Day 22 Notes on GenAI (Detailed)

1. Required Libraries

google-generativeai: Connects Python applications with Google's Gemini models for text generation, multimodal reasoning, summarization, and question answering.

sentence-transformers: Generates dense semantic embeddings where sentences with similar meanings are located close together in vector space. The notebook uses all-MiniLM-L6-v2, a lightweight yet highly accurate embedding model.

FAISS: Facebook AI Similarity Search. It stores millions of vectors and performs extremely fast nearest-neighbour searches. It is the core retrieval engine in many RAG systems.

NumPy: Stores embeddings as multidimensional arrays for efficient mathematical computation.

Streamlit: Builds interactive AI web applications with very little code.

pyngrok: Creates a public URL for a locally running Streamlit application.

2. Knowledge Base

A knowledge base is the external source of truth used by a Retrieval-Augmented Generation (RAG) system. Instead of relying only on the LLM's internal training data, the application retrieves relevant information from this knowledge base before generating an answer.

Best practices: Use clean and verified documents. Split large files into small chunks (200–1000 tokens). Store metadata such as title, source, page number and timestamps. Update documents regularly so answers remain current.

3. Embeddings

Embeddings convert text into high-dimensional numerical vectors. Unlike keyword search, embeddings capture semantic meaning. For example, 'automobile' and 'car' receive similar vectors even though the words differ.

Workflow: Load SentenceTransformer model. Encode each document. Obtain one vector per document. Store vectors in FAISS. Advantages include semantic search, multilingual capability, clustering and recommendation systems.

4. FAISS Vector Index

The embedding dimension is determined from the generated vectors. An IndexFlatL2 is created, which performs exact nearest-neighbour search using Euclidean (L2) distance.

Steps: Create the index. Add all document embeddings. Persist the index if required. Search using embedded user queries. For very large datasets, approximate indexes such as IVF or HNSW are preferred for better scalability.

5. Retrieval Process

When a user asks a question: The query is embedded using the same embedding model. FAISS compares the query vector with stored document vectors. The top-k most similar documents are returned. Retrieved context is appended to the prompt. Gemini generates an answer grounded in the retrieved information. This reduces hallucinations and improves factual accuracy.

6. Challenges in AI Interfaces

Common engineering problems include UI scalability, low-latency inference, asynchronous processing, performance optimization, caching, handling concurrent users, responsive design, accessibility, and maintaining conversational context.

7. Future of AI Interfaces

Future AI systems will increasingly use autonomous agents capable of planning and executing tasks, multimodal reasoning across text, images, audio and video, personalized assistants using user context, real-time workflows connected to enterprise software, and advanced semantic search powered by vector databases.

8. Responsible AI

Responsible AI focuses on privacy, fairness, transparency and accountability. Encrypt sensitive user data. Implement authentication and authorization. Audit datasets for bias. Explain model decisions where possible. Continuously monitor outputs for toxicity, misinformation and fairness.

9. Challenges in RAG

Major challenges: Retrieval latency. Poor chunking strategy. Low-quality embeddings. Duplicate documents. Large vector databases. Hallucinations despite retrieval. Solutions include optimized indexing, reranking models, hybrid search (keyword + vector), caching, better chunk sizes, metadata filtering and periodic re-indexing.

10. Real-World Applications

RAG powers enterprise knowledge assistants, customer support bots, educational tutors, document search, legal research, healthcare information systems, finance assistants, HR knowledge portals and internal company chatbots.

11. Key Takeaways

RAG combines retrieval with generation to produce more reliable answers. Streamlit simplifies deployment, embeddings provide semantic understanding, FAISS enables efficient similarity search, and Gemini produces context-aware responses.

12. Interview & Viva Notes

Difference between LLM and RAG, importance of embeddings, why FAISS is used, what chunking is, Top-K retrieval, vector databases, cosine similarity vs L2 distance, hallucination, context window, reranking, hybrid search, metadata filtering, and limitations of RAG are all frequently asked interview topics.

13. Complete RAG Pipeline

Documents → Cleaning → Chunking → Embedding Generation → Vector Database (FAISS) → User Query → Query Embedding → Similarity Search → Top-K Documents → Prompt Construction → Gemini LLM → Final Response → User

Quick Revision

Important Terms: RAG, Embedding, Vector Database, FAISS, Semantic Search, Chunking, Metadata, Context Window, Hallucination, Top-K Retrieval, L2 Distance, Cosine Similarity, Streamlit, Gemini API.

Remember: Better document quality + better chunking + better embeddings + better retrieval = better AI responses.